

Galgotias University

SCHOOL OF COMPUTING SCIENCE & ENGINEERING



LAB FILE
B.TECH
Python Programmimg
R1UC427B

Submitted by
Aman Patel

Submitted to :
Dr. Rajeshwari Sissodia

Admission No:
24SCSE1260003

Section : 50

PYTHON PROGRAMMING PRACTICAL – INDEX

S.No	Practical Name	Date	Progress	Teacher's Signature
1	Practical 1: Python Data Types & Variables	18 th February, 2026	Completed ✓	
2	Practical 2: Operators in Python	18 th February, 2026	Completed ✓	
3	Practical 3: Conditional Statements	25 th February, 2026	Completed ✓	
4	Practical 4: Looping Statements	25 th February, 2026	Completed ✓	
5	Practical 5: Pattern Printing	11 th March, 2026	Completed ✓	
6	Practical 6: Strings	11 th March, 2026	Completed ✓	
7	Practical 7: Lists	18 th March, 2026	Completed ✓	
8	Practical 8: Tuples	18 th March, 2026	Completed ✓	
9	Practical 9: Sets and Dictionary	25 th March, 2026	Completed ✓	

10	Practical 10: Functions	25 th March, 2026	Completed 	
11	Practical 11: Lambda Function, Modules & Math Module	1 st April, 2026	Completed 	
12	Practical 12: Creating Class and Object	15 th April, 2026	Completed 	
13	Practical 13: Constructors	22 th April, 2026	Completed 	
14	Practical 14: Inheritance & OOP Concepts	29 th April, 2026	Completed 	
15	Practical 15: NumPy & Pandas	6 th May, 2026	Completed 	
16	Practical 16: Mini Projects & Visualization	13 th May, 2026	Completed 	

EXPERIMENT 1

Aim: To understand Python variables and data types and perform basic variable operations.

Programmes:

1.1 Program to Print “Hello, World”

Aim: To write a basic Python program to print output.

Algorithm:

1. Start the program.
2. Use the print() function to display a message.
3. Stop the program.

Program:

```
print("Hello, World")  
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> Hello, World  
  
Aman Patel, 24SCSE1260003
```

1.2 Program to Declare and Print Different Types of Variables.

Aim: To declare variables of different data types and print them.

Algorithm:

1. Start the program.
 2. Declare integer, float, string, and Boolean variables.
 3. Use print() to display the values.
 4. Stop the program.
-



Program:

```
a = 10 # Integer
b = 20.5 # Float
c = "Python" # String
d = True # Boolean

print("Integer:", a)
print("Float:", b)
print("String:", c)
print("Boolean:", d)
print("Aman Patel, 24SCSE1260003")
```

Sample Output:

```
>>> Integer: 10  
  
Float: 20  
  
5 String: Python  
  
Boolean: True  
  
Aman Patel, 24SCSE1260003
```

1.3 Program to Take User Input and Display Its Data Type.

Aim: To accept input from the user and display its data type.

Algorithm:

1. Start the program.
2. Use `input()` to read a value from the user.
3. Display the value.
4. Use `type()` to display the data type.
5. Stop the program.

Program:

```
x = input("Enter a value: ")
print("Value:", x)
print("Data Type:", type(x))
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> Enter a value: 25

Value: 25

Data Type: <class 'str'>

Aman Patel, 24SCSE1260003
```



1.4 Program to Swap Two Variables.

Aim: To swap values of two variables.

Algorithm:

1. Start the program.
2. Assign values to two variables.
3. Display values before swapping.
4. Swap the values using Python multiple assignment.
5. Display values after swapping.
6. Stop the program.

Program:

```
a = 5
b = 10
print("Before swapping : ", a, b)
a, b = b, a
print("After swapping : ", a, b)
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> Before swapping : 5 10
      After swapping : 10 5
      Aman Patel, 24SCSE1260003
```

1.5 Program to Assign Multiple Variables in a Single Line.

Aim: To assign multiple variables in a single statement.

Algorithm:

1. Start the program.
2. Assign multiple values to variables in a single line.
3. Print the variable values.
4. Stop the program.



Program:

```
a, b, c = 1, 2, 3
```

```
print(a, b, c)
```

```
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> 1 2 3
```

```
Aman Patel, 24SCSE1260003
```

Result: Thus, the Python programs for printing output, declaring and printing different data types, taking user input and displaying its data type, swapping two variables, and assigning multiple variables in a single line were successfully executed and the outputs were verified.



EXPERIMENT 2

Aim: To understand and use different operators in Python.

Programmes:

2.1 Program Using Arithmetic Operators.

Aim: To perform arithmetic operations using Python operators.

Algorithm:

1. Start the program.
2. Assign values to two variables.
3. Perform addition, subtraction, multiplication, division, and modulus operations.
4. Display the results.
5. Stop the program.

Program:

```
a = 10
b = 5
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)
print("Modulus:", a % b)
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> Addition: 15
      Subtraction: 5
      Multiplication: 50
      Division: 2.0
      Modulus: 0
      Aman Patel, 24SCSE1260003
```



2.2 Program Using Relational Operators.

Aim: To compare two values using relational operators.

Algorithm:

1. Start the program.
2. Assign values to two variables.
3. Use relational operators to compare values.
4. Display the comparison results.
5. Stop the program.

Program:

```
a = 10
b = 5
print(a > b)
print(a < b)
print(a == b)
print(a != b)
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> True
False
False
True
Aman Patel, 24SCSE1260003
```

2.3 Program Using Logical Operators.

Aim: To use logical operators in Python.

Algorithm:

1. Start the program.
2. Assign Boolean values to variables.
3. Apply logical AND, OR, and NOT operators.
4. Display the results.
5. Stop the program.



Program:

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> False
True
False
Aman Patel, 24SCSE1260003
```

2.4 Program Using Assignment Operators.

Aim: To demonstrate assignment operators in Python.

Algorithm:

1. Start the program.
2. Assign an initial value to a variable.
3. Use assignment operators (+=, -=, *=, /=).
4. Display updated values.
5. Stop the program.

Program:

```
a = 10
a += 5
print(a)
a -= 3
print(a)
a *= 2
print(a)
a /= 4
print(a)
print("Aman Patel, 24SCSE1260003")
```

Output:

```
>>> 15 12 24 6.0
Aman Patel, 24SCSE1260003
```

Result: Thus, the Python programs using arithmetic, relational, logical, and assignment operators were successfully executed and the outputs were verified.



EXPERIMENT 3

Aim: To study and implement different types of conditional statements in Python.

Programmes:

3.1 Program using simple if statement.

Aim: To write a Python program using a simple if statement.

Algorithm:

1. Start the program.
2. Take an integer input from the user.
3. Check if the number is greater than 0.
4. If the condition is true, print "Number is positive".
5. Stop the program.

Program:

```
Practical 3 - Conditional Statements > 10_Simple_If.py > ...
1  num = int(input("Enter a number: "))
2
3  if num > 0:
4      print("Number is positive")
5
6  print(
7      "Name : Aman Patel\n"
8      "Admission Number : 24SCSE1260003\n"
9      "Batch : P2"
10 )
11
```

Output:

```
Enter a number: 123
Number is positive
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING> █
```

Conclusion:

Thus, the simple if statement was successfully implemented to execute a statement based on a given condition.



3.2 Program using if-else statement.

Aim: To write a Python program using if-else statement.

Algorithm:

1. Start the program.
2. Take an integer input from the user.
3. Check whether the number is even or odd.
4. If the number is divisible by 2, print "Even".
5. Otherwise, print "Odd".
6. Stop the program.

Program:

```
Practical 3 - Conditional Statements > 11_If_Else.py > ...
1  num = int(input("Enter a number: "))
2
3  if num % 2 == 0:
4      print("Even number")
5  else:
6      print("Odd number")
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
Enter a number: 324
Even number
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING>
```

Conclusion:

Thus, the if-else statement was successfully implemented to execute one of the two blocks depending on the given condition.



3.3 Program using nested if-else.

Aim: To write a Python program using nested if-else statements.

Algorithm:

1. Start the program.
2. Take an integer input from the user.
3. Check whether the number is greater than or equal to 0.
4. If the number is greater than or equal to 0.
5. Else if the number is negative.
6. Stop the program.

Program:

```
Practical 3 - Conditional Statements > 12_Nested_If_Else.py > ...
1  num = int(input("Enter a number: "))
2
3  if num >= 0:
4      if num == 0:
5          print("Number is zero")
6      else:
7          print("Number is positive")
8  else:
9      print("Number is negative")
10
11 print(
12     "Name : Aman Patel\n"
13     "Admission Number : 24SCSE1260003\n"
14     "Batch : P2"
15 )
16
```

Output:

```
Enter a number: 234
Number is positive
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING>
```

Conclusion:

Thus, the nested if-else statement was successfully implemented to check multiple conditions and produce the correct output.



3.4 Program using elif ladder.

Aim: To write a Python program using elif ladder.

Algorithm:

1. Start the program.
2. Take marks as input from the user.
3. Check marks range using if–elif ladder.
4. Print grade according to marks.
5. Stop the program.

Program:

```
Practical 3 - Conditional Statements > 13_Elif_Ladder.py > ...
1  marks = int(input("Enter marks: "))
2
3  if marks >= 90:
4      print("Grade A")
5  elif marks >= 75:
6      print("Grade B")
7  elif marks >= 50:
8      print("Grade C")
9  else:
10     print("Fail")
11
12 print(
13     "Name : Aman Patel\n"
14     "Admission Number : 24SCSE1260003\n"
15     "Batch : P2"
16 )
17
```

Output:

```
Enter marks: 98
Grade A
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING>
```

Result:

Thus, the programs based on conditional statements (simple if, if–else, nested if–else, and elif ladder) were successfully executed and the desired output was obtained.



EXPERIMENT 4

Aim: To study and implement different looping statements in Python.

Programmes:

4.1 Program using for loop.

Aim: To write a Python program using for loop.

Algorithm:

1. Start the program.
2. Use for loop to print numbers from 1 to 5.
3. Stop the program.

Program:

```
Practical 4 - Looping Statements > 14_For_Loop.py > ...
1  for i in range(1, 6):
2      print(i)
3
4  print(
5      "Name : Aman Patel\n"
6      "Admission Number : 24SCSE1260003\n"
7      "Batch : P2"
8  )
9
```

Output:

```
1
2
3
4
5
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING>
```

Conclusion:

Thus, the for loop was successfully implemented to execute statements repeatedly for a fixed range of values.



4.2 Program using while loop.

Aim: To write a Python program using while loop.

Algorithm:

1. Start the program.
2. Initialize a variable with value 1.
3. Use while loop to print numbers up to 5.
4. Increment the variable in each iteration.
5. Stop the program.

Program:

```
Practical 4 - Looping Statements > 15_While_Loop.py > ...
1  i = 1
2
3  while i <= 5:
4      print(i)
5      i += 1
6
7  print(
8      "Name : Aman Patel\n"
9      "Admission Number : 24SCSE1260003\n"
10     "Batch : P2"
11 )
12
```

Output:

```
1
2
3
4
5
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING>
```

Conclusion:

Thus, the while loop was successfully implemented to execute statements based on the given condition.



4.4 Program using break statement.

Aim: To write a Python program using break statement.

Algorithm:

1. Start the program.
2. Use loop from 1 to 10.
3. If number equals 5, break the loop.
4. Print numbers before break.
5. Stop the program.

Program:

```
Practical 4 - Looping Statements > 17_Break_Statement.py > ...
1  for i in range(1, 10):
2      if i == 5:
3          break
4      print(i)
5
6  print(
7      "Name : Aman Patel\n"
8      "Admission Number : 24SCSE1260003\n"
9      "Batch : P2"
10 )
11
```

Output:

```
1
2
3
4
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING>
```

Conclusion:

Thus, the break statement was successfully used to terminate the loop when the specified condition was satisfied.



4.5 Program using continue statement.

Aim: To write a Python program using continue statement.

Algorithm:

1. Start the program.
2. Use loop from 1 to 5.
3. If number equals 3, skip that iteration.
4. Print remaining numbers.
5. Stop the program.

Program:

```
Practical 4 - Looping Statements > 18_Continue_Statement.py > ...
1  for i in range(1, 6):
2      if i == 3:
3          continue
4      print(i)
5
6  print(
7      "Name : Aman Patel\n"
8      "Admission Number : 24SCSE1260003\n"
9      "Batch : P2"
10 )
11
```

Output:

```
1
2
4
5
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PYTHON PROGRAMMING>
```

Conclusion:

Thus, the continue statement was successfully used to skip a particular iteration and continue the execution of the loop.

Result:

Thus, the programs based on looping statements were successfully executed and the desired output was obtained.



EXPERIMENT 5

Aim: To study and implement different pattern printing programs using Python loops.

Programmes:

5.1 Program using continue statement.

Aim: To print a star pattern using Python.

Algorithm:

1. Start the program
2. Use a loop from 1 to 5
3. Print stars equal to loop value
4. Stop the program

Program:

```
Practical 5 - Pattern Printing > 19_Star_Pattern.py
1  rows = 4
2
3  for i in range(rows):
4      for j in range(rows):
5          print("*", end=" ")
6      print()
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
* * * *
* * * *
* * * *
* * * *

Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, the star pattern was printed successfully using loops.



5.2 Program to print number pattern.

Aim: Program to print number pattern.

Algorithm:

1. Start the program
2. Use nested loops
3. Print numbers from 1 to current row
4. Move to next line after each row
5. Stop the program.

Program:

```
Practical 5 - Pattern Printing > 20_Number_Pattern.py > ...
1   rows = 5
2
3   for i in range(1, rows + 1):
4       for j in range(1, i + 1):
5           print(j, end=" ")
6       print()
7
8   print(
9       "Name : Aman Patel\n"
10      "Admission Number : 24SCSE1260003\n"
11      "Batch : P2"
12  )
13
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, the number pattern was printed successfully.



5.3 Program to print number pattern.

Aim: Program to print pyramid pattern.

Algorithm:

1. Start the program
2. Use loop to control rows
3. Print spaces before stars
4. Print stars in increasing order
5. Stop the program.

Program:

```
Practical 5 - Pattern Printing > 21_Pyramid_Pattern.py > ...
1   rows = 5
2
3   for i in range(1, rows + 1):
4       print(" " * (rows - i), end="")
5       print("* " * i)
6
7   print(
8       "Name : Aman Patel\n"
9       "Admission Number : 24SCSE1260003\n"
10      "Batch : P2"
11  )
12
```

Output:

```

 *
 * *
 * * *
 * * * *
 * * * * *
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, the pyramid pattern was generated successfully.



5.4 Print alphabet triangle pattern.

Aim: To print alphabet triangle pattern using Python.

Algorithm:

1. Start the program
2. Use loop for rows
3. Print alphabets using ASCII values
4. Increment alphabet in each row
5. Stop the program.

Program:

```
Practical 5 - Pattern Printing > 22_Alphabet_Triangle.py > ...
1  rows = 5
2
3  for i in range(65, 65 + rows):
4      for j in range(65, i + 1):
5          print(chr(j), end=" ")
6      print()
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
A
A B
A B C
A B C D
A B C D E
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, the alphabet triangle pattern was printed successfully.



5.5 Program to print number pattern.

Aim: To print Pascal's triangle using Python.

Algorithm:

1. Start the program
2. Use loop for number of rows
3. Calculate values using formula
4. Print values in triangle format
5. Stop the program.

Program:

```
Practical 5 - Pattern Printing > 23_Pascals_Triangle.py > ...
1   rows = 5
2
3   for i in range(rows):
4       print(" " * (rows - i), end="")
5       num = 1
6       for j in range(i + 1):
7           print(num, end=" ")
8           num = num * (i - j) // (j + 1)
9       print()
10
11  print(
12      "Name : Aman Patel\n"
13      "Admission Number : 24SCSE1260003\n"
14      "Batch : P2"
15  )
16
```

Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, Pascal's triangle was printed successfully.

Result:

Thus, all the pattern printing programs were executed successfully and the desired outputs were obtained.



EXPERIMENT 6

Aim: To study and implement different string operations in Python.

Programmes:

6.1 Program to create and index a string.

Aim: To create a string and access its elements using indexing.

Algorithm:

1. Start the program
2. Declare a string
3. Access characters using index values
4. Display characters
5. Stop the program.

Program:

```
Practical 6 - Strings > 24_Create_Index_String.py > ...
1   s = "Python"
2
3   print("String:", s)
4   print("First character:", s[0])
5   print("Last character:", s[-1])
6
7   print(
8       "Name : Aman Patel\n"
9       "Admission Number : 24SCSE1260003\n"
10      "Batch: P2"
11  )
12
```

Output:

```
String: Python
First character: P
Last character: n
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, string elements were accessed successfully using indexing.



6.2 Program demonstrating string slicing.

Aim: To perform slicing operations on a string.

Algorithm:

1. Start the program
2. Declare a string
3. Use slicing to extract parts of string
4. Display sliced string
5. Stop the program.

Program:

```
Practical 6 - Strings > 25_String_Slicing.py > ...
1  s = "Programming"
2
3  print("Full string:", s[:])
4  print("First 5 characters:", s[:5])
5  print("From index 3 to 7:", s[3:8])
6  print("Last 4 characters:", s[-4:])
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
Full string: Programming
First 5 characters: Progr
From index 3 to 7: gramm
Last 4 characters: ming
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, string slicing was performed successfully.



6.3 Program using built-in string methods.

Aim: To use built-in string functions in Python.

Algorithm:

1. Start the program
2. Declare a string
3. Apply methods like upper(), lower(), etc.
4. Display results
5. Stop the program.

Program:

```
Practical 6 - Strings > 26_String_Methods.py > ...
1  s = "python programming"
2
3  print("Uppercase:", s.upper())
4  print("Lowercase:", s.lower())
5  print("Title case:", s.title())
6  print("Count of 'p':", s.count('p'))
7  print("Replace word:", s.replace("python", "Java"))
8
9  print(
10     "Name : Aman Patel\n"
11     "Admission Number : 24SCSE1260003\n"
12     "Batch : P2"
13 )
14
```

Output:

```
Uppercase: PYTHON PROGRAMMING
Lowercase: python programming
Title case: Python Programming
Count of 'p': 2
Replace word: Java programming
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, built-in string methods were used successfully.



6.4 Program to check palindrome string.

Aim: To check whether a string is palindrome or not.

Algorithm:

1. Start the program
2. Input a string
3. Reverse the string
4. Compare original and reversed string
5. Display result
6. Stop the program.

Program:

```
Practical 6 - Strings > 27_Palindrome_String.py > ...
1  s = input("Enter a string: ")
2
3  if s == s[::-1]:
4      print("Palindrome")
5  else:
6      print("Not Palindrome")
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
Enter a string: Scarce
Not Palindrome
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, palindrome string was checked successfully.



6.5 Program to find length of a string.

Aim: To find the length of a string.

Algorithm:

1. Start the program
2. Input a string
3. Use len() function
4. Display length
5. Stop the program.

Program:

```
Practical 6 - Strings > 28_String_Length.py > ...
1  s = input("Enter a string: ")
2  print("Length:", len(s))
3
4  print(
5      "Name : Aman Patel\n"
6      "Admission Number : 24SCSE1260003\n"
7      "Batch : P2"
8  )
9
```

Output:

```
Enter a string: scarce
Length: 6
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, the length of the string was found successfully.





EXPERIMENT 7

Aim: To study and implement various operations on lists in Python.

Programmes:

7.1 Program to create and access list elements.

Aim: To create a list and access its elements.

Algorithm:

1. Start the program
2. Create a list
3. Access elements using index
4. Display elements
5. Stop the program.

Program:

```
Practical 7 - Lists > 31_Create_Access_List.py > ...
1  nums = [10, 20, 30, 40, 50]
2
3  print("Full list:", nums)
4  print("First element:", nums[0])
5  print("Last element:", nums[-1])
6
7  print(
8      "Name : Aman Patel\n"
9      "Admission Number : 24SCSE1260003\n"
10     "Batch: P2"
11 )
12
```

Output:

```
Full list: [10, 20, 30, 40, 50]
First element: 10
Last element: 50
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING- JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, list elements were created and accessed successfully.



7.2 Program demonstrating list slicing.

Aim: To perform slicing on list elements.

Algorithm:

1. Start the program
2. Create a list
3. Apply slicing operation
4. Display result
5. Stop the program.

Program:

```
Practical 7 - Lists > 32_List_Slicing.py > ...
1  nums = [1, 2, 3, 4, 5, 6]
2
3  print("First 3 elements:", nums[:3])
4  print("From index 2 to 4:", nums[2:5])
5  print("Last 2 elements:", nums[-2:])
6
7  print(
8      "Name : Aman Patel\n"
9      "Admission Number : 24SCSE1260003\n"
10     "Batch : P2"
11 )
12
```

Output:

```
First 3 elements: [1, 2, 3]
From index 2 to 4: [3, 4, 5]
Last 2 elements: [5, 6]
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, list slicing was performed successfully.



7.3 Program using built-in list methods.

Aim: To use built-in list functions in Python.

Algorithm:

1. Start the program
2. Create a list
3. Apply methods like append(), remove(), etc.
4. Display updated list
5. Stop the program.

Program:

```
Practical 7 - Lists > 33_List_Methods.py > ...
1  nums = [1, 2, 3]
2
3  nums.append(4)
4  print("After append:", nums)
5
6  nums.insert(1, 10)
7  print("After insert:", nums)
8
9  nums.remove(2)
10 print("After remove:", nums)
11
12 nums.pop()
13 print("After pop:", nums)
14
15 print(
16     "Name : Aman Patel\n"
17     "Admission Number : 24SCSE1260003\n"
18     "Batch : P2"
19 )
20
```

Output:

```
After append: [1, 2, 3, 4]
After insert: [1, 10, 2, 3, 4]
After remove: [1, 10, 3, 4]
After pop: [1, 10, 3]
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, built-in list methods were used successfully.



7.4 Program to find sum of list elements.

Aim: To calculate sum of elements in a list.

Algorithm:

1. Start the program
2. Create a list
3. Use loop or sum()
4. Display sum
5. Stop the program.

Program:

```
Practical 7 - Lists > 34_Sum_Of_List.py > ...
1  nums = [10, 20, 30]
2  total = 0
3
4  for i in nums:
5      total += i
6
7  print("Sum:", total)
8
9  print(
10     "Name : Aman Patel\n"
11     "Admission Number : 24SCSE1260003\n"
12     "Batch : P2"
13 )
14
```

Output:

```
Sum: 60
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, the sum of list elements was calculated successfully.



7.4 Program to find largest and smallest element in a list.

Aim: To find maximum and minimum elements in a list.

Algorithm:

1. Start the program
2. Create a list
3. Use max() and min()
4. Display results
5. Stop the program.

Program:

```
Practical 7 - Lists > 35_Largest_Smallest_List.py > ...
1  nums = [45, 12, 78, 3, 56]
2
3  print("Largest:", max(nums))
4  print("Smallest:", min(nums))
5
6  print(
7      "Name : Aman Patel\n"
8      "Admission Number : 24SCSE1260003\n"
9      "Batch : P2"
10 )
11
```

Output:

```
Largest: 78
Smallest: 3
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, largest and smallest elements were found successfully.



7.6 Program to remove duplicate elements from a list.

Aim: To remove duplicate elements from a list.

Algorithm:

1. Start the program
2. Create a list
3. Convert list to set or use loop
4. Display unique elements
5. Stop the program.

Program:

```
Practical 7 - Lists > 36_Remove_Duplicates_List.py > ...
1  nums = [1, 2, 2, 3, 4, 4, 5]
2  unique = []
3
4  for i in nums:
5      if i not in unique:
6          unique.append(i)
7
8  print("After removing duplicates:", unique)
9
10 print(
11     "Name : Aman Patel\n"
12     "Admission Number : 24SCSE1260003\n"
13     "Batch : P2"
14 )
15
```

Output:

```
After removing duplicates: [1, 2, 3, 4, 5]
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, duplicate elements were removed successfully.



EXPERIMENT 8

Aim: To study and implement operations on tuples in Python.

Programmes:

8.1 Program to create and access tuple.

Aim: To create a tuple and access its elements.

Algorithm:

1. Start the program
2. Create a tuple
3. Access elements using index
4. Display elements
5. Stop the program.

Program:

```
Practical 8 - Tuples > 38_Create_Access_Tuple.py > ...
1  # 38. Program to create and access tuple
2
3  my_tuple = (10, 20, 30, 40)
4  print("Tuple:", my_tuple)
5  print("Second element:", my_tuple[1])
6
7  print(
8      "Name : Aman Patel\n"
9      "Admission Number : 24SCSE1260003\n"
10     "Batch : P2"
11 )
12
```

Output:

```
Tuple: (10, 20, 30, 40)
Second element: 20
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, tuple elements were accessed successfully.





8.3 Program to convert tuple to list.

Aim: To convert a tuple into a list.

Algorithm:

1. Start the program
2. Create a tuple
3. Convert using list()
4. Display result
5. Stop the program.

Program:

```
Practical 8 - Tuples > 40_Tuple_To_List.py > ...
1  # 40. Program to convert tuple to list
2
3  my_tuple = (1, 2, 3)
4  print("Tuple:", my_tuple, type(my_tuple))
5
6  my_list = list(my_tuple)
7  print("List:", my_list, type(my_list))
8
9  print(
10     "Name : Aman Patel\n"
11     "Admission Number : 24SCSE1260003\n"
12     "Batch : P2"
13 )
14
```

Output:

```
Tuple: (1, 2, 3) <class 'tuple'>
List: [1, 2, 3] <class 'list'>
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, tuple was converted to list successfully.



8.4 Program to find length of a tuple.

Aim: To find length of a tuple.

Algorithm:

1. Start the program
2. Create a tuple
3. Use len()
4. Display result
5. Stop the program.

Program:

```
Practical 8 - Tuples > 41_Length_Of_Tuple.py > ...
1  # 41. Program to find length of a tuple
2
3  my_tuple = ("a", "b", "c", "d")
4  print("Tuple:", my_tuple)
5  print("Length:", len(my_tuple))
6
7  print(
8      "Name : Aman Patel\n"
9      "Admission Number : 24SCSE1260003\n"
10     "Batch : P2"
11 )
12
```

Output:

```
Tuple: ('a', 'b', 'c', 'd')
Length: 4
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, length of tuple was found successfully.



8.5 Program to concatenate two tuples.

Aim: To concatenate two tuples.

Algorithm:

1. Start the program
2. Create two tuples
3. Use + operator
4. Display result
5. Stop the program.

Program:

```
Practical 8 - Tuples > 42_Concenate_Tuples.py > ...
1  # 42. Program to concatenate two tuples
2
3  tuple1 = (1, 2, 3)
4  tuple2 = (4, 5, 6)
5  combined = tuple1 + tuple2
6
7  print("Tuple 1:", tuple1)
8  print("Tuple 2:", tuple2)
9  print("Combined:", combined)
10
11 print(
12     "Name : Aman Patel\n"
13     "Admission Number : 24SCSE1260003\n"
14     "Batch : P2"
15 )
16
```

Output:

```
Tuple 1: (1, 2, 3)
Tuple 2: (4, 5, 6)
Combined: (1, 2, 3, 4, 5, 6)
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, tuples were concatenated successfully.

Result:

Thus, all tuple operations were executed successfully and correct outputs were obtained.



EXPERIMENT 9

Aim: To study and implement operations on sets and dictionaries in Python.

Programmes:

9.1 Program to perform set operations.

Aim: To perform basic operations like union, intersection on sets.

Algorithm:

1. Start the program
2. Create two sets
3. Perform union and intersection
4. Display results
5. Stop the program.

Program:

```
Practical 9 - Sets and Dictionary > 43_Set_Operations.py > ...
1  # 43. Program to perform set operations
2
3  set_a = {1, 2, 3, 4}
4  set_b = {3, 4, 5, 6}
5
6  print("Set A:", set_a)
7  print("Set B:", set_b)
8  print("Union:", set_a | set_b)
9  print("Intersection:", set_a & set_b)
10 print("Difference (A-B):", set_a - set_b)
11 print("Symmetric Difference:", set_a ^ set_b)
12
13 print(
14     "Name : Aman Patel\n"
15     "Admission Number : 24SCSE1260003\n"
16     "Batch : P2"
17 )
18
```

Output:

```
Set A: {1, 2, 3, 4}
Set B: {3, 4, 5, 6}
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (A-B): {1, 2}
Symmetric Difference: {1, 2, 5, 6}
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, set operations were performed successfully.



9.2 Program using set methods.

Aim: To use built-in methods of sets.

Algorithm:

1. Start the program
2. Create a set
3. Apply methods like add(), remove()
4. Display updated set
5. Stop the program.

Program:

```
Practical 9 - Sets and Dictionary > 44_Set_Methods.py > ...
1  # 44. Program using set methods
2
3  my_set = {1, 2, 3}
4  print("Original:", my_set)
5
6  my_set.add(4)
7  print("After add(4):", my_set)
8
9  my_set.remove(2)
10 print("After remove(2):", my_set)
11
12 my_set.discard(10) # No error if not present
13 print("After discard(10):", my_set)
14
15 my_set.clear()
16 print("After clear:", my_set)
17
18 print(
19     "Name : Aman Patel\n"
20     "AdmissionNumber : 24SCSE1260003\n"
21     "Batch : P2"
22 )
23
```

Output:

```
Original: {1, 2, 3}
After add(4): {1, 2, 3, 4}
After remove(2): {1, 3, 4}
After discard(10): {1, 3, 4}
After clear: set()
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, set methods were used successfully.



9.3 Program to create and print a dictionary.

Aim: To create and display a dictionary.

Algorithm:

1. Start the program
2. Create a dictionary with key-value pairs
3. Print the dictionary
4. Stop the program.

Program:

```
Practical 9 - Sets and Dictionary > 45_Create_Print_Dict.py > ...
1  # 45. Program to create and print a dictionary
2
3  student = {
4      "name": "John",
5      "age": 20,
6      "course": "CSE"
7  }
8  print("Dictionary:", student)
9  print(f"Name: {student['name']}")
10 print(f"Age: {student['age']}")
11
12 print(
13     "Name : Aman Patel\n"
14     "Admission Number : 24SCSE1260003\n"
15     "Batch : P2"
16 )
17
```

Output:

```
Dictionary: {'name': 'John', 'age': 20, 'course': 'CSE'}
Name: John
Age: 20
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING- JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, dictionary was created and printed successfully.



9.4 Program to add new key-value pairs to a dictionary.

Aim: To add elements to a dictionary.

Algorithm:

1. Start the program
2. Create a dictionary
3. Add new key-value pair
4. Display updated dictionary
5. Stop the program.

Program:

```
Practical 9 - Sets and Dictionary > 46_Add_Key_Value_Dict.py > ...
1  # 46. Program to add new key-value pairs to a dictionary
2
3  student = {"name": "John", "age": 20}
4  print("Original:", student)
5
6  student["city"] = "New York"
7  print("After adding city:", student)
8
9  print(
10     "Name : Aman Patel\n"
11     "Admission Number : 24SCSE1260003\n"
12     "Batch : P2"
13 )
14
```

Output:

```
Original: {'name': 'John', 'age': 20}
After adding city: {'name': 'John', 'age': 20, 'city': 'New York'}
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, new elements were added successfully.



9.6 Program to delete elements from a dictionary.

Aim: To delete elements from a dictionary.

Algorithm:

1. Start the program
2. Create a dictionary
3. Delete elements using del or pop()
4. Display updated dictionary
5. Stop the program.

Program:

```
Practical 9 - Sets and Dictionary > 48_Delete_Elements_Dict.py > ...
1  # 48. Program to delete elements from a dictionary
2
3  student = {"name": "John", "age": 20, "city": "New York"}
4  print("Original:", student)
5
6  del student["age"]
7  print("After deleting age:", student)
8
9  val = student.pop("city")
10 print(f"Popped value: {val}")
11 print("Final dictionary:", student)
12
13 print(
14     "Name : Aman Patel\n"
15     "Admission Number : 24SCSE1260003\n"
16     "Batch : P2"
17 )
18
```

Output:

```
Original: {'name': 'John', 'age': 20, 'city': 'New York'}
After deleting age: {'name': 'John', 'city': 'New York'}
Popped value: New York
Final dictionary: {'name': 'John'}
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, elements were deleted successfully.

Result:

Thus, all set and dictionary operations were executed successfully and correct outputs were obtained.



EXPERIMENT 10

Aim: To study and implement functions in Python.

Programmes:

10.1 Program to define and call a function.

Aim: To define and call a function.

Algorithm:

1. Start the program
2. Define a function
3. Call the function
4. Display output
5. Stop the program.

Program:

```
Practical 10 - Functions > 49_Define_Call_Function.py > ...
1  # 49. Program to define and call a function
2
3  def greet(name):
4      print(f"Hello, {name}!")
5
6  greet("Alice")
7  greet("Bob")
8
9  print(
10     "Name : Aman Patel\n"
11     "Admission Number : 24SCSE1260003\n"
12     "Batch : P2"
13 )
14
```

Output:

```
Hello, Alice!
Hello, Bob!
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING- JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, function was defined and executed successfully.



10.2 Program to check prime number using function.

Aim: To check whether a number is prime using a function.

Algorithm:

1. Start the program
2. Define function to check prime
3. Take input
4. Call function
5. Display result
6. Stop the program.

Program:

```
Practical 10 - Functions > 50_Check_Prime_Function.py > ...
1  # 50. Program to check prime number using function
2
3  def is_prime(n):
4      if n <= 1:
5          return False
6      for i in range(2, int(n ** 0.5) + 1):
7          if n % i == 0:
8              return False
9      return True
10
11  num = 29
12  if is_prime(num):
13      print(f"{num} is a prime number.")
14  else:
15      print(f"{num} is not a prime number.")
16
17  print(
18      "Name : Aman Patel\n"
19      "Admission Number : 24SCSE1260003\n"
20      "Batch : P2"
21  )
22
```

Output:

```
29 is a prime number.
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING- JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, prime number was checked successfully using function.



10.3 Program to find greatest of three numbers using function.

Aim: To find the greatest number among three using function.

Algorithm:

1. Start the program
2. Define function
3. Take three inputs
4. Compare numbers
5. Display greatest
6. Stop the program.

Program:

```
Practical 10 - Functions > 51_Greatest_Of_Three_Function.py > ...
1  def greatest_of_three(a, b, c):
2      if a >= b and a >= c:
3          return a
4      if b >= a and b >= c:
5          return b
6      return c
7
8
9  num1 = float(input("Enter first number: "))
10 num2 = float(input("Enter second number: "))
11 num3 = float(input("Enter third number: "))
12
13 greatest = greatest_of_three(num1, num2, num3)
14 print("Greatest number is:", greatest)
15 print(
16     "Name : Aman Patel\n"
17     "Admission Number : 24SCSE1260003\n"
18     "Batch : P2"
19 )
20
```

Output:

```
Enter first number: 245
Enter second number: 6536
Enter third number: 24525
Greatest number is: 24525.0
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, greatest number was found successfully.



10.4 Program to find sum of digits using function.

Aim: To calculate sum of digits of a number.

Algorithm:

1. Start the program
2. Define function
3. Take input number
4. Extract digits and add
5. Display result
6. Stop the program.

Program:

```
Practical 10 - Functions > 52_Sum_Digits_Function.py > ...
1  # 52. Program to find sum of digits using function
2
3  def sum_of_digits(n):
4      return sum(int(digit) for digit in str(n))
5
6  num = 12345
7  print(f"Number: {num}")
8  print(f"Sum of digits: {sum_of_digits(num)}")
9
10 print(
11     "Name : Aman Patel\n"
12     "Admission Number : 24SCSE1260003\n"
13     "Batch : P2"
14 )
15
```

Output:

```
Number: 12345
Sum of digits: 15
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, sum of digits was calculated successfully.



10.5 Program to check palindrome using function.

Aim: To check palindrome using function.

Algorithm:

1. Start the program
2. Define function
3. Take input
4. Reverse and compare
5. Display result
6. Stop the program.

Program:

```
Practical 10 - Functions > 53_Check_Palindrome_Function.py > ...
1  # 53. Program to check palindrome using function
2
3  def is_palindrome(s):
4      return s == s[::-1]
5
6  text = "racecar"
7  if is_palindrome(text):
8      print(f'{text} is a palindrome.')
9  else:
10     print(f'{text} is not a palindrome.')
11
12 print(
13     "Name : Aman Patel\n"
14     "Admission Number : 24SCSE1260003\n"
15     "Batch : P2"
16 )
17
```

Output:

```
'racecar' is a palindrome.
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, palindrome was checked successfully using function.



10.6 Program to perform calculator operations using functions.

Aim: To perform basic arithmetic operations using functions.

Algorithm:

1. Start the program
2. Define functions for operations
3. Take input from user
4. Perform operation
5. Display result
6. Stop the program.

Program:

```
Practical 10 - Functions > 54_Calculator_Function.py > ...
1  # 54. Program to perform calculator operations using functions
2
3  def add(x, y): return x + y
4  def subtract(x, y): return x - y
5  def multiply(x, y): return x * y
6  def divide(x, y): return x / y if y != 0 else "Error! Division by zero."
7
8  print("10 + 5 =", add(10, 5))
9  print("10 - 5 =", subtract(10, 5))
10 print("10 * 5 =", multiply(10, 5))
11 print("10 / 5 =", divide(10, 5))
12
13 print(
14     "Name : Aman Patel\n"
15     "Admission Number : 24SCSE1260003\n"
16     "Batch : P2"
17 )
18
```

Output:

```
10 + 5 = 15
10 - 5 = 5
10 * 5 = 50
10 / 5 = 2.0
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, calculator operations were performed successfully using functions.

Result:

Thus, all function-based programs were executed successfully and correct outputs were obtained.



EXPERIMENT 11

Aim: To study and implement lambda functions, modules, and math operations in Python.

Programmes:

11.1 Program to create and use a simple lambda function.

Aim: To create and use a lambda function.

Algorithm:

1. Start the program
2. Define a lambda function
3. Pass input value
4. Display result
5. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 55_Lambda_Function.py > ...
1  # 55. Program to create and use a simple lambda function
2
3  square = lambda x: x * x
4  print("Square of 5 is:", square(5))
5
6  print(
7      "Name : Aman Patel\n"
8      "Admission Number : 24SCSE1260003\n"
9      "Batch : P2"
10 )
11
```

Output:

```
Square of 5 is: 25
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, lambda function was created and used successfully.



11.2 Program to find square of a number using lambda function.

Aim: To calculate square using lambda function.

Algorithm:

1. Start the program
2. Define lambda for square
3. Input number
4. Display result
5. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 56_Square_Lambda.py > ...
1  # 56. Program to find square of a number using lambda function
2
3  # Similar to 55, but specifically focusing on square as per question list
4  calc_square = lambda n: n ** 2
5  num = 6
6  print(f"Square of {num} is {calc_square(num)}")
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
Square of 6 is 36
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, square was calculated successfully using lambda.



11.3 Program to add two numbers using lambda function.

Aim: To add two numbers using lambda function.

Algorithm:

1. Start the program
2. Define lambda for addition
3. Input numbers
4. Display sum
5. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 57_Add_Lambda.py > ...
1  # 57. Program to add two numbers using lambda function
2
3  add = lambda a, b: a + b
4  print("Sum of 10 and 20 is:", add(10, 20))
5
6  print(
7      "Name : Aman Patel\n"
8      "Admission Number : 24SCSE1260003\n"
9      "Batch : P2"
10 )
11
```

Output:

```
Sum of 10 and 20 is: 30
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, addition was performed successfully using lambda.



11.4 Program to import a module and use its functions.

Aim: To import a module and use its functions.

Algorithm:

1. Start the program
2. Import module
3. Use module function
4. Display result
5. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 58_Import_Module.py
1  # 58. Program to import a module and use its functions
2
3  import math
4  print("Sqrt of 16:", math.sqrt(16))
5
6  print(
7      "Name : Aman Patel\n"
8      "Admission Number : 24SCSE1260003\n"
9      "Batch : P2"
10 )
11
```

Output:

```
Sqrt of 16: 4.0
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, module was imported and used successfully.



11.5 Program to use math module constants (pi, e).

Aim: To use constants from math module.

Algorithm:

1. Start the program
2. Import math module
3. Access constants like pi and e
4. Display values
5. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 59_Math_Constants.py
1  # 59. Program to use math module constants (pi, e)
2
3  import math
4
5  print("Pi:", math.pi)
6  print("Euler's number (e):", math.e)
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
Pi: 3.141592653589793
Euler's number (e): 2.718281828459045
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, math constants were accessed successfully.



11.6 Program to perform square root and power operations using math module.

Aim: To perform square root and power calculations.

Algorithm:

1. Start the program
2. Import math module
3. Use sqrt() and pow()
4. Display results
5. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 60_Sqrt_Power_Math.py > ...
1  # 60. Program to perform square root and power operations using math module
2
3  import math
4
5  num = 25
6  print(f"Square root of {num}: ", math.sqrt(num))
7  print(f"2 raised to power 3: ", math.pow(2, 3))
8
9  print(
10     "Name : Aman Patel\n"
11     "Admission Number : 24SCSE1260003\n"
12     "Batch : P2"
13 )
14
```

Output:

```
Square root of 25: 5.0
2 raised to power 3: 8.0
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, square root and power operations were performed successfully.



11.7 Program to find factorial using math module.

Aim: To calculate factorial using math module.

Algorithm:

1. Start the program
2. Import math module
3. Input number
4. Use factorial()
5. Display result
6. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 61_Factorial_Math.py > ...
1  # 61. Program to find factorial using math module
2
3  import math
4
5  num = 5
6  print(f"Factorial of {num}:", math.factorial(num))
7
8  print(
9      "Name : Aman Patel\n"
10     "Admission Number : 24SCSE1260003\n"
11     "Batch : P2"
12 )
13
```

Output:

```
Factorial of 5: 120
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, factorial was calculated successfully.



11.8 Program to use trigonometric functions (sin, cos, tan).

Aim: To use trigonometric functions from math module.

Algorithm:

1. Start the program
2. Import math module
3. Input angle
4. Apply sin(), cos(), tan()
5. Display results
6. Stop the program.

Program:

```
Practical 11 - Lambda and Modules > 62_Trig_Functions.py > ...
1  # 62. Program to use trigonometric functions (sin, cos, tan)
2
3  import math
4
5  angle = 45
6  radians = math.radians(angle)
7
8  print(f"Sin({angle}):", math.sin(radians))
9  print(f"Cos({angle}):", math.cos(radians))
10 print(f"Tan({angle}):", math.tan(radians))
11
12 print(
13     "Name : Aman Patel\n"
14     "Admission Number : 24SCSE1260003\n"
15     "Batch : P2"
16 )
17
```

Output:

```
Sin(45): 0.7071067811865476
Cos(45): 0.7071067811865476
Tan(45): 0.9999999999999999
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, trigonometric functions were executed successfully.

Result:

Thus, all programs using lambda functions and modules were executed successfully and correct outputs were obtained.



EXPERIMENT 12

Aim: To study and implement classes and objects in Python.

Programmes:

12.1 Program to create a simple class and object.

Aim: To create a class and instantiate an object.

Algorithm:

1. Start the program
2. Define a class
3. Create object
4. Access attributes
5. Display output
6. Stop the program.

Program:

```
Practical 12 - Class and Object > 63_Simple_Class_Object.py > ...
1  # 63. Program to create a simple class and object
2
3  class Person:
4      def __init__(self, name):
5          self.name = name
6
7      def greet(self):
8          print(f"Hello, my name is {self.name}")
9
10 p = Person("Alice")
11 p.greet()
12
13 print(
14     "Name : Aman Patel\n"
15     "Admission Number : 24SCSE1260003\n"
16     "Batch : P2"
17 )
18
```

Output:

```
Hello, my name is Alice
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING- JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, class and object were created successfully.



12.2 Program to create a student class and display details.

Aim: To create a student class and display student information.

Algorithm:

1. Start the program
2. Define student class
3. Add attributes like name, roll no
4. Create object
5. Display details
6. Stop the program.

Program:

```
1  # 64. Program to create a student class and display details
2
3  class Student:
4      def __init__(self, name, roll_no):
5          self.name = name
6          self.roll_no = roll_no
7
8      def display(self):
9          print(f"Student Name: {self.name}")
10         print(f"Roll No: {self.roll_no}")
11
12     s = Student("Bob", 101)
13     s.display()
14
15     print(
16         "Name : Aman Patel\n"
17         "Admission Number : 24SCSE1260003\n"
18         "Batch : P2"
19     )
20
```

Output:

```
Student Name: Bob
Roll No: 101
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, student details were displayed successfully using class.



12.3 Program to create an employee class and calculate salary.

Aim: To create employee class and calculate salary.

Algorithm:

1. Start the program
2. Define employee class
3. Add salary attributes
4. Calculate total salary
5. Display result
6. Stop the program.

Program:

```
Practical 12 - Class and Object > 65_Employee_Class.py > ...
1  # 65. Program to create an employee class and calculate salary
2
3  class Employee:
4      def __init__(self, name, salary):
5          self.name = name
6          self.salary = salary
7
8      def calculate_bonus(self):
9          return self.salary * 0.10
10
11  emp = Employee("Charlie", 50000)
12  print(f"Employee: {emp.name}")
13  print(f"Salary: {emp.salary}")
14  print(f"Bonus: {emp.calculate_bonus()}")
15
16  print(
17      "Name : Aman Patel\n"
18      "Admission Number : 24SCSE1260003\n"
19      "Batch : P2"
20  )
21
```

Output:

```
Employee: Charlie
Salary: 50000
Bonus: 5000.0
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, employee salary was calculated successfully.

Result:

Thus, all class and object programs were executed successfully and correct outputs were obtained.



EXPERIMENT 13

Aim: To study and implement constructors in Python.

Programmes:

13.1 Program using default constructor.

Aim: To create and use a default constructor.

Algorithm:

1. Start the program
2. Define a class with default constructor
3. Initialize values
4. Create object
5. Display output
6. Stop the program.

Program:

```
Practical 13 - Constructors > 66_Default_Constructor.py > ...
1  # 66. Program using default constructor
2
3  class Student:
4      def __init__(self):
5          self.name = "Unknown"
6          self.age = 0
7          print("Default constructor called")
8
9      def display(self):
10         print(f"Name: {self.name}, Age: {self.age}")
11
12     s = Student()
13     s.display()
14
15     print(
16         "Name : Aman Patel\n"
17         "Admission Number : 24SCSE1260003\n"
18         "Batch : P2"
19     )
20
```

Output:

```
Default constructor called
Name: Unknown, Age: 0
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING- JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, default constructor was implemented successfully.



13.2 Program using parameterized constructor.

Aim: To create and use a parameterized constructor.

Algorithm:

1. Start the program
2. Define class with constructor parameters
3. Pass values while creating object
4. Display output
5. Stop the program.

Program:

```
Practical 13 - Constructors > 67_Parameterized_Constructor.py > ...
1  # 67. Program using parameterized constructor
2
3  class Student:
4      def __init__(self, name, age):
5          self.name = name
6          self.age = age
7          print("Parameterized constructor called")
8
9      def display(self):
10         print(f"Name: {self.name}, Age: {self.age}")
11
12     s = Student("Alice", 20)
13     s.display()
14
15     print(
16         "Name : Aman Patel\n"
17         "Admission Number : 24SCSE1260003\n"
18         "Batch : P2"
19     )
20
```

Output:

```
Parameterized constructor called
Name: Alice, Age: 20
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, parameterized constructor was implemented successfully.

Result:

Thus, constructor-based programs were executed successfully.



EXPERIMENT 14

Aim: To study and implement inheritance in Python.

Programmes:

14.1 Program demonstrating single inheritance.

Aim: To demonstrate single inheritance.

Algorithm:

1. Start the program
2. Create parent class
3. Create child class inheriting parent
4. Access parent properties
5. Display output
6. Stop the program.

Program:

```
Practical 14 - Inheritance and OOP > 68_Single_Inheritance.py > ...
1  # 68. Program demonstrating single inheritance
2
3  class Animal:
4      def speak(self):
5          print("Animal speaks")
6
7  class Dog(Animal):
8      def bark(self):
9          print("Dog barks")
10
11  d = Dog()
12  d.speak()
13  d.bark()
14
15  print(
16      "Name : Aman Patel\n"
17      "Admission Number : 24SCSE1260003\n"
18      "Batch : P2"
19  )
20
```

Output:

```
Animal speaks
Dog barks
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, single inheritance was implemented successfully.



14.2 Program demonstrating multilevel inheritance.

Aim: To demonstrate multilevel inheritance.

Algorithm:

1. Start the program
2. Create base class
3. Create derived class
4. Create another class inheriting derived
5. Display output
6. Stop the program.

Program:

```
Practical 14 - Inheritance and OOP > 69_Multilevel_Inheritance.py > ...
1  # 69. Program demonstrating multilevel inheritance
2
3  class Animal:
4      def speak(self):
5          print("Animal speaks")
6
7  class Dog(Animal):
8      def bark(self):
9          print("Dog barks")
10
11 class Puppy(Dog):
12     def weep(self):
13         print("Puppy weeps")
14
15 p = Puppy()
16 p.speak()
17 p.bark()
18 p.weep()
19
20 print(
21     "Name : Aman Patel\n"
22     "Admission Number : 24SCSE1260003\n"
23     "Batch : P2"
24 )
25
```

Output:

```
Animal speaks
Dog barks
Puppy weeps
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, multilevel inheritance was implemented successfully.



14.3 Program demonstrating multiple inheritance.

Aim: To demonstrate multiple inheritance.

Algorithm:

1. Start the program
2. Create multiple parent classes
3. Create child class inheriting them
4. Access methods
5. Display output
6. Stop the program.

Program:

```
Practical 14 - Inheritance and OOP > 70_Multiple_Inheritance.py > ...
1  # 70. Program demonstrating multiple inheritance
2
3  class Father:
4      def gardening(self):
5          print("Father enjoys gardening")
6
7  class Mother:
8      def cooking(self):
9          print("Mother enjoys cooking")
10
11 class Child(Father, Mother):
12     def playing(self):
13         print("Child enjoys playing")
14
15 c = Child()
16 c.gardening()
17 c.cooking()
18 c.playing()
19
20 print(
21     "Name : Aman Patel\n"
22     "Admission Number : 24SCSE1260003\n"
23     "Batch : P2"
24 )
25
```

Output:

```
Father enjoys gardening
Mother enjoys cooking
Child enjoys playing
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, multiple inheritance was implemented successfully.



14.4 Program demonstrating encapsulation.

Aim: To demonstrate encapsulation in Python.

Algorithm:

1. Start the program
2. Create class with private variables
3. Use methods to access data
4. Display output
5. Stop the program.

Program:

```
Practical 14 - Inheritance and OOP > 71_Encapsulation.py > ...
1  # 71. Program demonstrating encapsulation
2
3  class BankAccount:
4      def __init__(self, balance):
5          self.__balance = balance # Private variable
6
7      def deposit(self, amount):
8          if amount > 0:
9              self.__balance += amount
10             print(f"Deposited: {amount}")
11
12     def get_balance(self):
13         return self.__balance
14
15 acc = BankAccount(1000)
16 acc.deposit(500)
17 print(f"Balance: {acc.get_balance()}")
18 # print(acc.__balance) # This would raise AttributeError
19
20 print(
21     "Name : Aman Patel\n"
22     "Admission Number : 24SCSE1260003\n"
23     "Batch : P2"
24 )
25
```

Output:

```
Deposited: 500
Balance: 1500
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, encapsulation was implemented successfully.



14.6 Program demonstrating method overriding.

Aim: To demonstrate method overriding.

Algorithm:

1. Start the program
2. Create parent class
3. Create child class
4. Override method
5. Call method
6. Display output
7. Stop the program.

Program:

```
Practical 14 - Inheritance and OOP > 73_Method_Overriding.py > ...
1  # 73. Program demonstrating method overriding
2
3  class Animal:
4      def make_sound(self):
5          print("Animal makes a sound")
6
7  class Dog(Animal):
8      def make_sound(self):
9          print("Dog barks")
10
11  d = Dog()
12  d.make_sound()
13
14  print(
15      "Name : Aman Patel\n"
16      "Admission Number : 24SCSE1260003\n"
17      "Batch : P2"
18  )
19
```

Output:

```
Dog barks
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, method overriding was implemented successfully.

Result:

Thus, all object-oriented programming concepts including constructors, inheritance, encapsulation, abstraction, and polymorphism were successfully implemented and verified.



EXPERIMENT 15

Aim: To study and implement NumPy, Pandas, and data visualization in Python.

Programmes:

15.1 Program to create NumPy array.

Aim: To create and use a NumPy array.

Algorithm:

1. Start the program
2. Import NumPy module
3. Create an array
4. Display the array
5. Stop the program.

Program:

```
Practical 15 - NumPy Pandas Matplotlib > 74_Create_Numpy_Array.py > ...
1  # 74. Program to create NumPy array
2
3  import numpy as np
4
5  arr = np.array([1, 2, 3, 4, 5])
6  print("NumPy Array:", arr)
7  print("Type:", type(arr))
8  print("Shape:", arr.shape)
9
10 print(
11     "Name : Aman Patel\n"
12     "Admission Number : 24SCSE1260003\n"
13     "Batch : P2"
14 )
15
```

Output:

```
NumPy Array: [1 2 3 4 5]
Type: <class 'numpy.ndarray'>
Shape: (5,)
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, NumPy array was created successfully.



15.2 Program to create Pandas Series.

Aim: To create a Pandas Series.

Algorithm:

1. Start the program
2. Import Pandas module
3. Create a Series
4. Display the Series
5. Stop the program.

Program:

```
Practical 15 - NumPy Pandas Matplotlib > 75_Create_Pandas_Series.py > ...
1  # 75. Program to create Pandas Series
2
3  import pandas as pd
4
5  data = [10, 20, 30, 40]
6  series = pd.Series(data, index=['a', 'b', 'c', 'd'])
7  print("Pandas Series:")
8  print(series)
9
10 print(
11     "Name : Aman Patel\n"
12     "Admission Number : 24SCSE1260003\n"
13     "Batch : P2"
14 )
15
```

Output:

```
Pandas Series:
a    10
b    20
c    30
d    40
dtype: int64
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, Pandas Series was created successfully.



15.3 Program to plot line graph, bar chart, pie chart.

Aim: To plot graphs using Matplotlib.

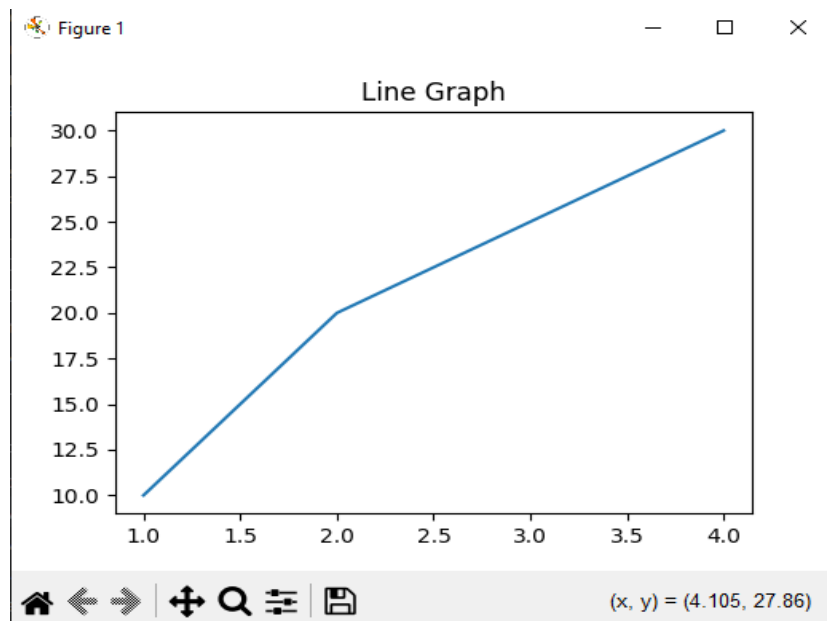
Algorithm:

1. Start the program
2. Import matplotlib
3. Prepare data
4. Plot line, bar, and pie chart
5. Display graphs
6. Stop the program.

Program:

```
Practical 15 - NumPy Pandas Matplotlib > 76_Line_Bar_Pie_Chart.py > ...
1  # 76. Program to plot line graph, bar chart, pie chart
2
3  import matplotlib.pyplot as plt
4
5  # Line Graph
6  x = [1, 2, 3, 4]
7  y = [10, 20, 25, 30]
8  plt.figure()
9  plt.plot(x, y)
10 plt.title("Line Graph")
11 plt.show()
12
13 # Bar Chart
14 categories = ['A', 'B', 'C']
15 values = [10, 15, 7]
16 plt.figure()
17 plt.bar(categories, values)
18 plt.title("Bar Chart")
19 plt.show()
20
21 # Pie Chart
22 sizes = [30, 20, 50]
23 plt.figure()
24 plt.pie(sizes, labels=categories, autopct='%1.1f%%')
25 plt.title("Pie Chart")
26 plt.show()
27
28 print(
29     "Name : Aman Patel\n"
30     "Admission Number : 24SCSE1260003\n"
31     "Batch : P2"
32 )
33
```

Output:



Conclusion:

Thus, graphs were plotted successfully using Matplotlib.

Result:

Thus, data handling and visualization programs were executed successfully.



EXPERIMENT 16

Aim: To implement real-world applications using Python concepts.

Programmes:

16.1 Student Class Management System.

Aim: To create a student management system using class, object, and constructor.

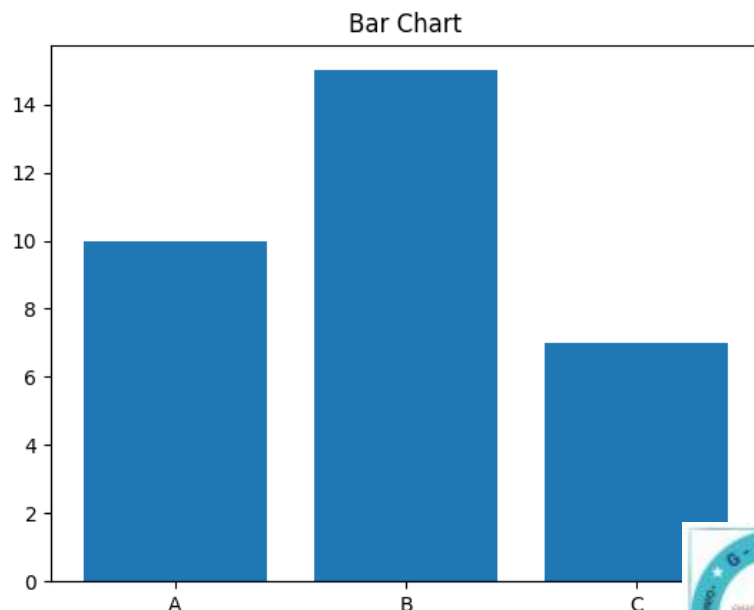
Algorithm:

1. Start the program
2. Define student class
3. Initialize details using constructor
4. Create object
5. Display student details
6. Stop the program.

```
Practical 16 - Mini Projects > 78_Student_Management_System.py > ...
1 # 78. Student Class Management System: Use class, object, constructor.
2
3 class Student:
4     def __init__(self, roll_no, name, marks):
5         self.roll_no = roll_no
6         self.name = name
7         self.marks = marks
8
9     def display(self):
10        print(f"Roll No: {self.roll_no}, Name: {self.name}, Marks: {self.marks}")
11
12 students = []
13 students.append(Student(1, "Alice", 85))
14 students.append(Student(2, "Bob", 90))
15 students.append(Student(3, "Charlie", 78))
16
17 print("Student Details:")
18 for s in students:
19     s.display()
20
21 print(
22     "Name : Aman Patel\n"
23     "Admission Number : 24SCSE1260003\n"
24     "Batch : P2"
25 )
26
```

Output:

Figure 1



Conclusion:

Thus, student management system was implemented successfully.



16.2 Library Management System.

Aim: To create a library system using inheritance.

Algorithm:

1. Start the program
2. Create base class (Book)
3. Create derived class (Member)
4. Use inheritance
5. Display details
6. Stop the program.

Program:

```
Practical 16 - Mini Projects > 79_Library_Management_System.py > ...
1  # 79. Library Management System: Use inheritance for books and members.
2
3  class LibraryMember:
4      def __init__(self, member_id, name):
5          self.member_id = member_id
6          self.name = name
7
8      def display_member(self):
9          print(f"Member ID: {self.member_id}, Name: {self.name}")
10
11 class Book(LibraryMember):
12     def __init__(self, member_id, name, book_title):
13         super().__init__(member_id, name)
14         self.book_title = book_title
15
16     def display_details(self):
17         super().display_member()
18         print(f"Borrowed Book: {self.book_title}")
19
20 b = Book(101, "David", "Python Programming")
21 b.display_details()
22
23 print(
24     "Name : Aman Patel\n"
25     "Admission Number : 24SCSE1260003\n"
26     "Batch : P2"
27 )
28
```

Output:

```
Member ID: 101, Name: David
Borrowed Book: Python Programming
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, library management system was implemented successfully using inheritance.



16.3 Shape Area Calculator.

Aim: To calculate area using method overriding.

Algorithm:

1. Start the program
2. Create base class
3. Create child classes for shapes
4. Override area method
5. Display result
6. Stop the program.

Program:

```
Practical 16 - Mini Projects > 80.Shape_Area_Calculator.py > ...
1  # 80. Shape Area Calculator: Use method overriding for different shapes
2
3  class Shape:
4      def area(self):
5          print("Calculating area...")
6
7  class Rectangle(Shape):
8      def __init__(self, l, w):
9          self.l = l
10         self.w = w
11
12         def area(self):
13             return self.l * self.w
14
15  class Circle(Shape):
16      def __init__(self, r):
17          self.r = r
18
19         def area(self):
20             return 3.14 * self.r * self.r
21
22  r = Rectangle(10, 5)
23  c = Circle(7)
24
25  print(f"Rectangle Area: {r.area()}")
26  print(f"Circle Area: {c.area()}")
27
28  print(
29      "Name : Aman Patel\n"
30      "Admission Number : 24SCSE1260003\n"
31      "Batch : P2"
32  )
33
```

Output:

```
Rectangle Area: 50
Circle Area: 153.86
Name : Aman Patel
Admission Number : 24SCSE1260003
Batch : P2
PS C:\Users\User\Desktop\SCARCE\PROGRAMMING-JOURNEY\#UNIVERSITY\PYTHON>
```

Conclusion:

Thus, area was calculated successfully using method overriding.



16.4 Attendance Visualization.

Aim: To visualize attendance data using Matplotlib.

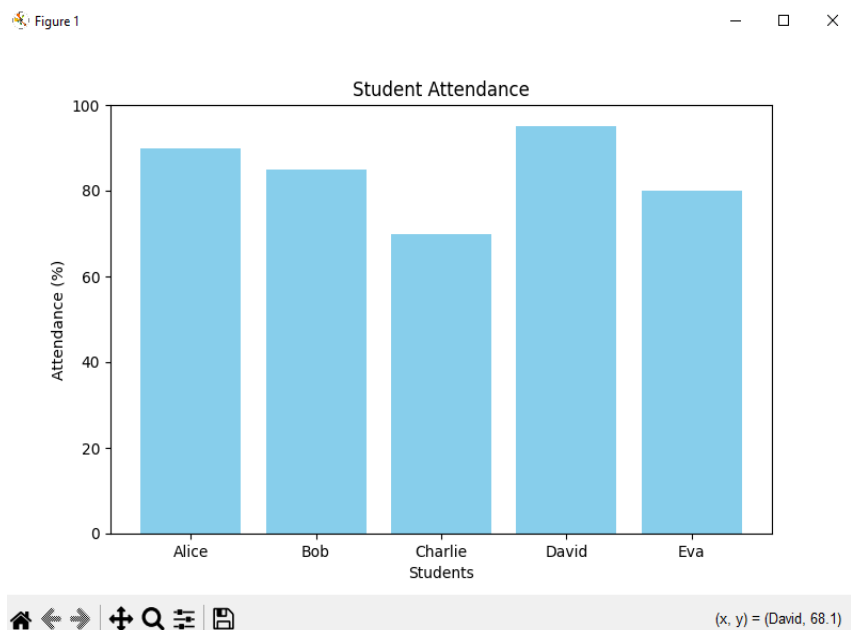
Algorithm:

1. Start the program
2. Import matplotlib
3. Prepare attendance data
4. Plot graph
5. Display visualization
6. Stop the program.

Program:

```
Practical 15 - NumPy Pandas Matplotlib > 77_Attendance_Visualization.py > ...
1  # 77. Attendance Visualization: Plot attendance using Matplotlib.
2
3  import matplotlib.pyplot as plt
4
5  students = ['Alice', 'Bob', 'Charlie', 'David', 'Eva']
6  attendance = [90, 85, 70, 95, 80]
7
8  plt.figure(figsize=(8, 5))
9  plt.bar(students, attendance, color='skyblue')
10 plt.xlabel('Students')
11 plt.ylabel('Attendance (%)')
12 plt.title('Student Attendance')
13 plt.ylim(0, 100)
14 plt.show()
15
16 print(
17     "Name : Aman Patel\n"
18     "Admission Number : 24SCSE1260003\n"
19     "Batch : P2"
20 )
21
```

Output:



Conclusion:

Thus, attendance data was visualized successfully.

Result:

Thus, all practical programs and mini projects were successfully implemented and verified, demonstrating the understanding of Python programming concepts.

